

linnest

a typst interface to the `linnet` crate

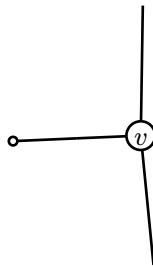
`linnet` is a rust crate that provides a graph data structure and many graph algorithms, and crucially for this package, multiple layout algorithms, as well as a dot parser (using the `dot_parser` crate).

Contents

Linnet	1
Linnest	1
Minimal Build Example	2
Graph Objects	3
Graph Specs	3
Placements	5
Physics Edge Styles	6
Graph Queries	8
Layout Model	8
Subgraphs	10
Generated Reference	10
graph	10
layout	32
physics	39
draw	48
subgraph	58

Linnest

The `linnet` crate that this package wraps around, is built around the half-edge graph data structure. This means that instead of a graph being represented as a set of nodes and edges, it is represented as a set of half-edges H , and a set of vertices V . The graph structure is then encoded through two maps. The first map, $\partial : H \rightarrow V$ maps each half-edge to its corresponding vertex. The preimage of any vertex v is the set of half-edges that map to it, called the crown of v . The second map, $\iota : H \rightarrow H$, is an involution that *glues* half edges together to form edges. If a half-edge is glued to itself, we call that an external half-edge. This means that `linnet` graphs are strictly more capable than normal edge and vertex graphs.



A side effect of natively supporting half-edges, is that subgraphs can be more granular, as we can encode them as sets of half-edges. This naturally supports vertex induced subgraphs (the union of the crowns of a set of vertices).

Linnest

Linnest is the `typst` plugin + API to integrate with `linnet` and access some of its capabilities. This means layouting algorithms, and dot parsing (all through `wasm`, so no dependencies/ external tooling required) along with a selection of graph algorithms (that can be expanded upon as needed).

Graphs can be constructed in two ways, parsing from a dot string using `parse`:

```
#let g = parse("digraph { a -> b }")
```

or building from edges and nodes using build, with a fletcher inspired syntax:

```
#let g = build({
  node(<a>, label: [$v$])
  node(<b>)
  edge(source(<a>), <a-b>, sink(<b>), label: [e])
  edge(source(<a>), <in-a1>, label: [e])
  edge(source(<a>), <in-a2>, label: [e])
})
```

In either case, type(g)=dictionary, because graph values are Typst dictionaries that wrap an archived linnet graph plus native Typst data. Rust owns topology, layout state, statement metadata, and internal opaque payload bytes; user data captured from Typst stays in Typst and is merged back into query records.

The main usecase however is of course to automatically place the nodes and edges on a canvas, using the layout function.

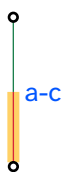
- graph for construction, parsing, inspection, joins, and graph algorithms.
- subgraph for subgraph object construction and inspection.
- layout for the separate layout pass.
- draw for rendering a laid-out graph object with CeTZ.
- physics for reusable particle-line edge styles.

Minimal Build Example

```
#import "../src/lib.typ": draw, graph, layout, subgraph
#import graph: build, dot, edge, edges, node, nodes, parse, sink, source
```

```
#let g = build({
  node(<a>)
  node(<c>)
  edge(
    source(<a>, compass: "e"),
    <a-c>,
    sink(<c>, compass: "w"),
    label: [a-c],
    statements: (
      color: "0055ff",
      source-color: "d72638",
      sink-color: "1b7f4c",
    ),
  ),
},
name: "demo",
)

#let g = layout(g)
#let east = subgraph.compass(g, "e")
#let edge-records = edges(g, subgraph: east)
#let dot-text = dot(g)
#let edge-label(edge) = text(fill: rgb("#" + edge.color))[#edge.label]
#let source-style(edge) = (stroke: rgb("#" + edge.source-color) + 0.5pt)
#let sink-style(edge) = (stroke: rgb("#" + edge.sink-color) + 0.5pt)
#draw(g, subgraph: east, edge-label: edge-label, source-style: source-style, sink-style: sink-style)
```



Graph Objects

Graph objects are Typst dictionaries wrapping archived Rust graph bytes plus native Typst data arrays for graph, node, edge, source, and sink data. The Rust graph may also carry an internal opaque payload, but that payload is used only by the Typst wrapper and is not exposed in public records. Build or parse graph objects with `graph`, transform graph objects with `layout`, and pass objects back to `graph` or `subgraph` for inspection. Subgraph objects are still opaque zero-copy values.

- `graph.parse(input)` parses one or more DOT digraphs and returns an array of graph objects. Its `eval-graph-fields`, `eval-node-fields`, `eval-edge-fields`, `eval-source-fields`, and `eval-sink-fields` arguments are convenience arguments for `graph.eval-fields`.
- `graph.build(...)` constructs one graph object from a stream of node and edge items.
- `graph.map(graph, ...)` maps graph, node, edge, source, and sink records to new native data without changing topology.
- `graph.node-data(graph, <name>)` and `graph.edge-data(graph, <name>)` return one named node or edge data.
- `graph.update-node-data(graph, <name>, data)` and `graph.update-edge-data(graph, <name>, data)` update one named node or edge data. Direct replacements and callbacks run in Typst with `(data, record)`.
- `graph.eval-fields(graph, ...)` evaluates selected record fields into data entries. It works on parsed and built graph objects.
- `node(...)` returns a node item.
- `source(...)` and `sink(...)` return half-edge endpoints.
- `edge(...)` returns an edge item built from source/sink endpoints and an optional edge name.
- `layout(graph, ...)` runs layout as an explicit second step. Its settings are named parameters so calls stay descriptive and Tidy can document each field.
- `draw(graph, ...)` draws a laid-out graph object with CeTZ.
- `dot(graph)` returns a DOT string for inspection or export.

Graph Specs

The Typst construction API is half-edge first: create node items, create source and sink half-edge endpoints, then pass edge items to `graph.build`. `graph.build` accepts both comma-separated items and ordinary Typst code blocks. Typst labels such as `<a>`, `<h1>`, and `<e1>` are API names; node and edge names are emitted back from `nodes(g)` and `edges(g)` as Typst labels. They are resolved before the wire format is sent to the Rust plugin. Numeric `id` arguments choose graph indexes or ordering: on nodes, `id` fixes the resulting node index and must be unique and in bounds. On nodes, edges, sources, and sinks, extra named arguments are captured as opaque Typst data fields: `edge(source(<a>, style: physics.source-stroke()), sink(<b>), particle: "g")` stores `(style: ...)` in the source data and `(particle: "g")` in the edge data. These user data values are not sent through the Rust plugin boundary. Typst sends an internal opaque payload with correlation data, asks Rust to resolve the graph indexes, then stores user data in arrays at those resolved graph, node, edge, and half-edge ids. `graph.info`, `graph.nodes`, and `graph.edges` merge those native values into the returned records. A captured `label` data field on nodes and edges is display content used by the default drawing style; use statements: `(label: "...")` when a flat metadata label string is needed. Statements are flat metadata used by DOT; they cannot nest. Values are scalar strings/numbers/booleans. Use data fields for structured Typst data or content.

`graph.build` and `graph.parse` also accept `default-node-data`, `default-edge-data`, `default-source-data`, and `default-sink-data`. These defaults are merged into the corresponding data; captured data fields on nodes, edges, sources, and sinks override the defaults. For drawing, the physics helpers read `data.style` on source and sink half-edges and `data.display-label/data.label` on edges:

```
#let g = build({
  node(<a>, label: [a])
  node(<b>, label: [b])
  edge(
    source(<a>, style: physics.source-stroke(c: red)),
    <e>,
    sink(<b>, style: physics.sink-stroke(c: blue)),
```

```

    label: [$p$],
    particle: "g",
  )
},
default-edge-data: (kind: "propagator"),
)
#let callbacks = physics.style()
#draw(
  layout(g),
  source-style: callbacks.source-style,
  sink-style: callbacks.sink-style,
  edge-label: callbacks.edge-label,
)

```

When parsing DOT, the same native data arrays can be filled from selected string fields. Rust parses DOT into topology and statement metadata; Typst applies defaults and evaluates selected fields afterward. The selected fields are evaluated with the record's merged `fields` dictionary in scope. Since node names are labels, use `#str(name)` when a name should become visible text:

```

#let g = parse(
  "digraph g { a [label=\"A\"]; a -> b [label=\"$p$\", source=\"out\", sink=\"in\"] }",
  eval-node-fields: ("label"),
  eval-edge-fields: ("label"),
  eval-source-fields: ("statement"),
  eval-sink-fields: ("statement"),
).first()
#nodes(g).first().data.label
#edges(g).first().data.label

```

The same transform can run after construction. This is useful for global edge statements that should apply to every edge while still seeing local edge fields:

```

#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), sink(<b>), statements: (mom: "p"))
}, default-edge-statements: (display-label: "$#mom$"))
#let g = graph.eval-fields(g, eval-edge-fields: ("display-label"))
#edges(g).first().data.at("display-label")

#let g = build({
  node(<a>)
  node(<c>)
  edge(source(<a>), <e1>, sink(<c>))
})

```

Named nodes and edges can be updated after construction without scanning in the caller. The update replaces the native data, or it can be a callback receiving `(data, record)`:

```

#let g = build({
  node(<a>)
  node(<c>)
  edge(source(<a>), <e1>, sink(<c>))
})
#let g = update-node-data(g, <a>, (label: [A]))
#let g = update-edge-data(g, <e1>, (data, edge) => (
  label: [$p$],
  source: edge.source.node,
))
#node-data(g, <a>).label
#edge-data(g, <e1>).label

```

`source(..)` and `sink(..)` accept a node reference plus optional name, id, statement, and compass. name is a Typst label name for the half-edge; id is numeric:

```

edge(source(<a>, name: <h1>, id: 0), <e1>, sink(<c>, id: 2), label: [a-c])

```

One half-edge creates an external edge. The side is determined by the constructor, so there is no public flow argument:

```
edge(<incoming>, sink(<a>))
edge(source(<c>), <outgoing>)
```

graph.build does not interpolate statement strings on the Rust side. Use graph.eval-fields or graph.map when a default statement should turn into Typst content or structured data data. The evaluation scope includes the record's merged fields, so default edge statements can still refer to local edge fields:

```
#let g = build({
  node(<a>)
  node(<c>)
  edge(
    source(<a>),
    <a-c>,
    sink(<c>),
    label: [a-c],
    statements: (color: "0055ff", label: "a-c"),
  )
},
default-edge-statements: (
  color: "000000",
  display-label: "$#label$",
),
)
#let g = graph.eval-fields(g, eval-edge-fields: ("display-label",))
```

Placements

graph.pos creates a first-class placement. The default mode: "pin" turns a coordinate into a fixed layout constraint and a drawable position. Use mode: "start" when the coordinate should only seed the layout:

```
#let g = build({
  node(<a>, pos: graph.pos(x: -2, y: 0))
  node(<c>, pos: graph.pos(ref: <a>, dx: 4, dy: 0))
  edge(source(<a>), <a-c>, sink(<c>), pos: graph.pos(x: 0, y: 1.2))
})
```

graph.group links one coordinate across several nodes or edge control points. A side of "+" keeps the coordinate positive, and "-" keeps it negative. GammaLoop external-edge columns use this to keep incoming and outgoing external legs on opposite sides while pairing rows by a shared y group:

```
#let edge-items = (
  edge(
    source(<right-ext>),
    sink(<center>),
    pos: graph.pos(x: graph.group("right", side: "+"), y: graph.group("edgee0")),
  ),
  edge(
    source(<center>),
    sink(<left-ext>),
    pos: graph.pos(x: graph.group("left", side: "-"), y: graph.group("edgee0")),
  ),
)
```

Raw DOT input uses the same placement model through the pos attribute. The standard Graphviz subset is preserved: pos="x,y" is a starting coordinate and pos="x,y!" is pinned. Linnest extends the value with explicit id references and axis entries. In axis entries, ! belongs to that axis: numeric entries without ! are starting coordinates, numeric entries with ! are fixed constraints, and grouped entries must use ! because groups are constraints.

```
digraph {
  a [id=0 pos="0,0!"]
  b [id=1 pos="ref(node:0)+4,0!"]
  a -> b [id=0 pos="ref(node:1)+0,1!"]
}
```

```

b -> c [id=1 pos="ref(edge:0)+1,0!"]
c [id=2 pos="x:2!"]
d [id=3 pos="x:2!,y:1"]
ext -> a [id=2 pos="x:@-left!,y:@edge0!"]
}

```

The DOT grammar is intentionally id-based: `ref(node:0)` uses a node id and `ref(edge:0)` uses an edge id. Bare names and implicit edge order are not placement references. The `@` syntax denotes grouped coordinate constraints; `@+name` and `@-name` keep the grouped coordinate on the positive or negative side respectively.

```

pos="x,y"           // start x and y
pos="x,y!"          // pin x and y
pos="x:<coord>"      // start numeric x
pos="x:<coord>!"     // pin x
pos="y:<coord>!"     // pin y
pos="x:<coord>!,y:<coord>" // pin x, start numeric y
pos="x:<coord>,y:<coord>!" // start numeric x, pin y

```

Draw styling is Typst-native. Pass dictionaries or callbacks to `draw`; edge callbacks receive the merged scope, edge statements, source/sink half-edge records, and the edge index:

```

#let edge-label(edge) = text(fill: rgb("#" + edge.color))[#edge.display-label]
#let source-style(edge) = (stroke: red + 0.5pt)
#let sink-style(edge) = (stroke: blue + 0.5pt)
#draw(layout(g), edge-label: edge-label, source-style: source-style, sink-style: sink-style)

```

Marks can follow graph orientation as a first-class draw option. Put the same mark layer on both halves and set `mark-orientation: "edge"`; default-oriented edges mark the source half, reversed edges mark the sink half with the marker flipped, and undirected edges suppress the mark.

```

#let oriented-arrow = (
  stroke: black + 0.7pt,
  mark: (end: (symbol: ">", fill: black, anchor: "center", shorten-to: auto), scale: 0.75),
  mark-position: "center-if-dangling",
  mark-orientation: "edge",
)
#draw(layout(g), source-style: oriented-arrow, sink-style: oriented-arrow)

```

Physics Edge Styles

`physics.style(...)` returns `source-style`, `sink-style`, and `edge-label` callbacks for `draw`. The default source/sink strokes use darker/lighter halves to encode the graph source/sink split. Fermion map entries marked with `fermion-flow` receive one particle-flow arrow on the main edge using `mark-orientation: "edge"`; paired edges follow `edge.orientation`, dangling half edges place the arrow in the middle of the visible half edge, and undirected edges omit the arrow. This is independent of momentum arrows.

Set `momentum-arrows: true` to draw centered parallel arrow decorations while the main edge remains connected to the nodes. The arrows flow from source to sink independently of the DOT `dir/orientation` value; there is one momentum marker per edge, even though the base edge is internally styled as source and sink halves. On paired edges the marker lives on the sink half; on dangling edges it lives on the existing source or sink half-edge. The decoration defaults to a plain black 0.55pt stroke and a scaled CeTZ "barbed" arrowhead, so it does not inherit the base particle stroke. The arrow length is capped by both `momentum-arrow-length` and `momentum-arrow-ratio`, so the shorter one wins. When the layout provides an `edge-label` position, the arrow offset is signed so the momentum marker is drawn on the same side of the edge as that label. Use `momentum-arrow-offset` to set the normal displacement. `momentum-arrow-mark: auto` uses the default barbed marker, `momentum-arrow-mark: none` draws only the momentum line, and any other value overrides the CeTZ mark style, for example "stealth" or `(end: "barbed", scale: 1.6)`. Only the end mark is kept so source/sink splitting cannot create a second momentum head.

Optional labels can be built from edge metadata with `show-edge-index`, `show-half-edge-index`, `show-particle`, and, for explicit momentum fields, `show-momentum`. `show-half-edge-index` only emits a value for dangling half edges.

```

#import "../src/lib.typ": draw, graph, layout, physics
#import graph: build, dot, edge, edges, node, nodes, parse, sink, source

#let g = build({
  node(<a>)
  node(<c>)
  edge(
    source(<a>),
    <fermion-main>,
    sink(<c>),
    id: 7,
    particle: "fermion",
  )
  edge(
    source(<c>),
    <fermion-out>,
    id: 8,
    particle: "fermion",
  )
},
name: "physics",
)
#let callbacks = physics.style(
  momentum-arrows: true,
  show-edge-index: true,
  show-particle: true,
)
#draw(
  layout(g),
  source-style: callbacks.source-style,
  sink-style: callbacks.sink-style,
  edge-label: callbacks.edge-label,
)

```

Set edge-offset on draw, or offset in a source-style/sink-style dictionary, to draw a fitted parallel path. Style callbacks may also return an array of dictionaries; the layers are drawn in order on the same graph edge, so parallel strokes do not require duplicate graph edges.

```

#let base-style(edge) = (
  stroke: (paint: gray, thickness: 0.7pt, cap: "round"),
)
#let offset-style(edge) = (
  offset: 0.18,
  length: 1.4,
  ratio: 0.5,
  resolve-length: "min",
  stroke: (paint: rgb("#2f6f4e"), thickness: 1pt, cap: "round"),
)
#let source-style(edge) = (base-style(edge), offset-style(edge))
#let sink-style(edge) = (base-style(edge), offset-style(edge) + (mark: (end: ">")))

```

The parallel path is applied to the base edge geometry before patterns and other decorations; node outlets then trim the shifted path, so shifted paths still start and end outside fitted node circles. Add edge-length or length to center-trim the shifted path to a fixed arc length, and add edge-ratio or ratio to cap it by a fraction of the base edge length. edge-resolve-length

resolve-length decides how to combine both limits "min"/"shorter" (default), "max"/"longer",

"length"/"fixed", "ratio"/"relative", "none"/"full", or a function receiving (base-length, length, ratio). Set offset-side: "label" on an offset layer to choose the sign of offset so the layer is drawn on the same side of the curve as the edge label.

Data defaults are also the global styling hook for all sources, sinks, nodes, and edges. More specific data can be added with captured named arguments on node, edge, source, and sink items, or by running

graph.map/graph.eval-fields after construction. This keeps evaluated Typst values in the native data channel instead of adding renderer-specific eval fields to the Rust topology spec:

```
#let g = build({
  node(<a>)
  node(<c>)
  edge(
    source(<a>),
    <a-c>,
    sink(<c>),
    label: [a-c],
    kind: "highlight",
  )
},
name: "demo",
default-source-data: (style: (stroke: red + 0.5pt)),
default-sink-data: (style: (stroke: blue + 0.5pt)),
)
#let callbacks = physics.style()
#draw(layout(g), source-style: callbacks.source-style, sink-style: callbacks.sink-style)
```

graph.build also accepts comma-separated items:

```
#let g = build(
  node(<a>),
  node(<b>),
  edge(
    source(<a>, compass: "e"),
    <ab>,
    sink(<b>, compass: "w"),
    label: [ab],
    statements: (color: "0055ff", label: "ab"),
  ),
name: "demo",
statements: (full_num: "x + y"),
default-node-statements: (shape: "circle"),
default-edge-statements: (
  color: "000000",
  display-label: "$#label$",
),
)
```

Graph Queries

graph.info(g) returns graph metadata. nodes(g) returns node records, and edges(g) returns edge records. Node and edge record name values are Typst labels when present. Pass subgraph: sg to filter nodes or edges by an subgraph object.

graph.join(left, right, key: "statement") joins matching dangling half edges. The key is read from half-edge statements or numeric ids and can be "statement", "compass", or "id".

graph.cycles(g) returns subgraph objects for a cycle basis. graph.forests(g) returns subgraph objects for spanning forests.

Layout Model

layout starts from a traversal-tree placement, then optimizes the positions of graph nodes and edge control points. The initial tree spacing is

$$L = \lambda \sqrt{\frac{WH}{\max(n, 1)}}$$

,

with horizontal spacing $\tau_x L$ and vertical spacing $\tau_y L$. Here λ is length-scale, W is viewport-w, H is viewport-h, τ_x is tree-dx, and τ_y is tree-dy. These fields set the geometry scale for both layout modes.

For deterministic, non-iterative placement, use `layout-algo: "tree"` or `layout-algo: "dot"`. "tree" places a traversal forest by levels. "dot" uses a directed layered placement for acyclic inputs, falling back to tree placement when the selected edges contain a cycle. Both modes also work on a subgraph:

```
#let g = parse("digraph partial { a -> b; b -> c; c -> d; d -> a }").at(0)
#let tree = graph.forests(g).at(0)
#let g = layout(g, layout-algo: "tree", subgraph: tree)
#draw(g)
```

Only nodes touched by the selected subgraph are placed. Edge control points are then resolved from the current node positions, so edges outside the selected subgraph are drawn as straight lines unless their own position constraints say otherwise.

`layout-algo: "force"` and `layout-algo: "anneal"` also accept subgraph. For these iterative modes, nodes and edge control points outside the selected subgraph stay fixed and act as boundary points while the selected subgraph is optimized.

Set `layout-nodes: "fixed"` to keep every node at its current pos for this layout pass and move only edge control points. With subgraph, only edges in the selected subgraph are moved; all other edge control points keep their current positions. The fixed-node policy is temporary; the returned graph stores the resulting coordinates as `pos`, but it does not turn them into persistent `pin` constraints. This is useful after one node-placement pass when a later pass should route or relax selected edges without disturbing the node layout:

```
#let g = parse("digraph partial { a [pos=\"0,0\"]; b [pos=\"4,0\"]; c [pos=\"8,0\"]; a -> b; b -> c }").at(0)
#let first = subgraph.bits(g, (true, true, false, false))
#let g = layout(g, layout-algo: "tree", layout-nodes: "fixed", subgraph: first)
#draw(g)
```

The shared spring/charge model uses the following coefficients:

$$\begin{aligned} c_{vv} &= \beta L^2 \\ c_{ev} &= \beta \gamma_{ev} L^2 \\ c_{ee} &= \beta \gamma_{ee} L^2 \\ c_{\text{center}} &= \beta g_{\text{center}} L^2 \\ c_{\text{dangling}} &= \beta \gamma_{\text{dangling}} L^2 \end{aligned}$$

The Typst parameter names are β for β , γ_{ev} for γ_{ev} , γ_{ee} for γ_{ee} , g_{center} for g_{center} , and γ_{dangling} for γ_{dangling} . The spring stiffness k is `k-spring`, and the softening constant ε is `eps`.

In `layout-algo: "anneal"`, linnest minimizes an energy:

$$\begin{aligned} E = & \sum_{i < j} \frac{1}{2} \frac{c_{vv}}{d(v_i, v_j) + \varepsilon} + \sum_{i, e} \frac{c_{ev}}{d(v_i, e) + \varepsilon} + \sum_{(v, e) \text{ incident}} \frac{1}{2} k (\ell_e - d(v, e))^2 \\ & + \sum_{\text{local edge pairs}} \frac{1}{2} \frac{c_{ee}}{d(e_i, e_j) + \varepsilon} + \sum_{\text{dangling pairs}} \frac{1}{2} \frac{c_{\text{dangling}}}{d(e_i, e_j) + \varepsilon} + \sum_i \frac{1}{2} \frac{c_{\text{center}}}{d(v_i, 0) + \varepsilon} + p_{\text{cross}} N_{\text{cross}} \end{aligned}$$

.

Here p_{cross} is crossing-penalty and N_{cross} is the number of detected edge crossings. `temp`, `step`, `seed`, `steps`, `epochs`, `cool`, `accept-floor`, `step-shrink`, and `incremental-energy` belong to this simulated annealing mode. crossing-penalty is also `anneal-only`; force mode does not currently add a crossing force.

In `layout-algo: "force"`, linnest applies the direct forces corresponding to the same vertex-vertex, edge-vertex, incidence spring, local edge-edge, dangling-edge, and center terms. `step` is the integration step, `delta clamps per-step movement`, `steps` and `epochs` set the iteration budget, `cool` shrinks the step after each epoch, and `early-tol` stops when movement is small. `z-spring` and `z-spring-growth` are force-only helpers: the integrator gives points temporary `z` coordinates to break overlaps and pulls them back toward the 2D plane.

directional-force is applied in both modes as an extra bias derived from pin/port direction constraints.

After either graph layout mode, labels are relaxed separately. If L_l is the label target distance and q_l is the label repulsion strength, then $L_l = \alpha_l L$ and $q_l = \beta_l L^2$. The Typst names are label-length-scale for α_l and label-charge for β_l . label-spring is the spring constant pulling each label toward its target. label-steps, label-step, label-early-tol, and label-max-delta-scale control the label relaxation iteration.

Subgraphs

Subgraph objects are opaque zero-copy values.

- `subgraph.label(g, label)` constructs a subgraph from a base62 label.
- `subgraph.bits(g, bits)` constructs a subgraph from a boolean hedge array.
- `subgraph.compass(g, compass)` selects half edges with a DOT compass point.
- `subgraph.to-label(sg)` returns the base62 label.
- `subgraph.hedges(sg)` returns included hedge indices.
- `subgraph.contains(sg, hedge)` tests hedge membership.

Generated Reference

graph

- `build()`
- `parse()`
- `node()`
- `source()`
- `sink()`
- `edge()`
- `group()`
- `pos()`
- `map()`
- `eval-fields()`
- `info()`
- `dot()`
- `nodes()`
- `edges()`
- `node-data()`
- `edge-data()`
- `update-node-data()`
- `update-edge-data()`
- `join()`
- `cycles()`
- `forests()`

build

Build one graph object from a stream of node and edge items.

Use `node()`, `source()`, `sink()`, and `edge()` to create graph items. Positional items may be passed as comma-separated arguments or yielded from a Typst code block.

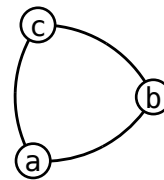
Parameters

```
build(  
  ..items: array,  
  name: none string,  
  data: any,  
  statements: dictionary,  
  default-node-statements: dictionary,  
  default-edge-statements: dictionary,  
  default-node-data: any,  
  default-edge-data: any,  
  default-source-data: any,  
  default-sink-data: any  
) -> dictionary
```

..items array

Node and edge items returned by `node()` and `edge()`. The best way to use these is to use a code scope, so that the edges and nodes append each other:

```
#let g = build({  
  node(<a>, label: [a])  
  node(<b>, label: [b])  
  node(<c>, label: [c])  
  edge(source(<a>), sink(<b>))  
  edge(source(<b>), sink(<c>))  
  edge(source(<a>), sink(<c>))  
})
```



name none or string

Graph name.

```
#let g = build(name: "My Graph", {  
  })  
#info(g).name
```

My Graph

Default: none

data any

Native Typst graph data.

```
#let g = build(data: (a:(b:(1,))), {  
  })  
#info(g).data
```

(a: (b: (1,)))

Default: none

statements dictionary

Flat graph statements, that get turned into a string to string dictionary in rust. Used by DOT parsing; values cannot nest.

```
#let g = build(statements: (a:1), {  
  })  
#info(g).global-statements
```

```
(a: "1")
```

Default: (:)

default-node-statements dictionary

Flat default node statements. Used by DOT; values cannot nest.

Default: (:)

default-edge-statements dictionary

Flat default edge statements. Used by DOT parsing; values cannot nest. These are applied/delegated to all edges.

```
#let g = build(default-edge-statements: (a:1),  
  {  
    node(<a>  
    edge(source(<a>))  
    edge(sink(<a>))  
  })  
#edges(g).map(e=>e.statements)
```

```
((a: "1"), (a: "1"))
```

Default: (:)

default-node-data any

Default data merged into every node data. Captured node data fields override it.

```
#let g = build(default-node-data: (a:1), {  
  node(<a>  
  node(<b>, a:2)  
  node(<c>, b:(a:1))  
  })  
#nodes(g).map(n=>n.data)
```

```
((a: 1), (a: 2), (a: 1, b: (a: 1)))
```

Default: none

default-edge-data any

Default data merged into every edge data. Captured edge data fields override it.

```
#let g = build(default-edge-data: (a:1,b:
[$m_mu$]), {
  node(<a>,id:0)
  edge(source(<a>),id:1)
  edge(sink(<a>),<e>,id:0,b:[#set
text(font:"Reforma")
  This is a test: ])
})
#edges(g).map(e=>e.data.b).join()
```

This is a test: m_μ

Default: **none**

default-source-data any

Default data merged into every source half-edge data. Captured source data fields override it.

```
#let g = build(default-source-data: (a:1), {
  node(<a>)
  edge(source(<a>))
  edge(sink(<a>))
})
#edges(g).map(e=>if e.source != none
{e.source.data} else {none})
```

((a: 1), **none**)

Default: **none**

default-sink-data any

Default data merged into every sink half-edge data. Captured sink data fields override it.

```
#let g = build(default-sink-data: (a:1), {
  node(<a>)
  edge(source(<a>))
  edge(sink(<a>))
})
#edges(g).map(e=>if e.sink != none
{e.sink.data} else {none})
```

(**none**, (a: 1))

Default: **none**

parse

Parse one or more DOT graphs into graph objects.

Default data are applied before eval-* fields are evaluated, so default data strings can refer to parsed record fields such as #str(name). Parsed fields take precedence over default data fields, so DOT label="..." overrides default-node-data: (label: ...).

Linnest uses dot-parser for DOT syntax and then gives special meaning to a small set of attributes. Every other attribute is preserved as a flat string statement. Preserved statements are visible through info(), nodes(), edges(), map(), and eval-fields(), but they do not become native Typst data unless a matching eval-* argument is passed.

```
#let src = ``dot
digraph { a [label="A"]; }
```


```
#let n = nodes(parse(src.text).first()).first()
#n.statements.label
#n.data
```


```

A

#### Graph-level handling:

- The DOT graph name becomes `graph.info(g).name`. A graph-level name attribute can name an anonymous graph; a named graph/digraph header takes precedence.

```
#let src = ``dot
digraph header_wins { graph [name="ignored"];
a }
```


```
#info(parse(src.text).first()).name
```


```

header_wins

- Top-level key=value statements and `graph [key=value]` attributes become `graph.info(g).global-statements`, except for the consumed name. They do not configure layout, even when a key looks like a layout option; pass layout options directly to `layout`.

```
#let src = ``dot
digraph { steps=1; graph [subtitle="nested"];
a }
```


```
#let statements =
info(parse(src.text).first()).global-statements
#statements.steps
#statements.subtitle
```


```

1 nested

- `node [key=value]` and `edge [key=value]` become default-node-statements and default-edge-statements. They are merged into each parsed node or edge before local attributes, so local attributes override defaults.

```
#let src = ``dot
digraph { node [color=gray]; edge [particle=g];
a [color=red]; a -> b }
```


```
#let g = parse(src.text).first()
#nodes(g).first().statements.color
#edges(g).first().statements.particle
```


```

red g

Node handling:

- The DOT node id becomes the node name returned by `nodes()`. If the DOT node id is numeric, it is consumed as the node index instead and the public name is none.

```
#let src = ``dot
digraph { alpha; 1; }
```


```
#nodes(parse(src.text).first()).map(n => n.name)
```


```

(<alpha>, none)

- `id=<n>` is also consumed as an explicit node index. Explicit node indexes must be unique and in bounds after parsing.

```
#let src = ``dot
digraph { a [id=1]; b [id=0]; }
```


```
#nodes(parse(src.text).first()).map(n => n.name)
```


```

(, <a>)

- `style=invis` marks the DOT node as a dangling external endpoint. It is not returned by `nodes()`. Its remaining attributes are copied onto the external edge that touches it, except `shape` and `label`; `style` and `id` are consumed.

```
#let src = ``dot
digraph { ext [style=invis, column=left,
label=skip]; ext -> a [id=0]; }
```


```
#let g = parse(src.text).first()
#nodes(g).map(n => n.name)
#edges(g).first().statements.column
#edges(g).first().statements.at("label",
default: none)
```


```

```
(<a>,) left
```

- Node `style` is consumed for every node; only `style=invis` has special behavior. Use another attribute name for drawing style metadata that must survive parsing.

```
#let src = ``dot
digraph { a [style=filled, "draw-
style=filled"]; }
```


```
#nodes(parse(src.text).first()).first().statements
```


```

```
("draw-style": "filled")
```

- `pos` is parsed as node placement. Simple numeric values are exposed as `node.pos`; extended placement values are used by `layout`. `pin` is an explicit layout constraint and is not exposed as a public statement.

```
#let src = ``dot
digraph { a [id=0, pos="0,0!"]; b [id=1,
pos="ref(node:0)+2,0!"]; }
```


```
#let parsed-nodes =
nodes(parse(src.text).first())
#parsed-nodes.map(n => n.pos)
#parsed-nodes.map(n => n.statements.at("pin",
default: none))
```


```

```
(none, none) (none, none)
```

- `shift` is parsed as a drawing shift and also remains available as a statement. `eval` is retained for legacy round-tripping. Other node attributes are preserved as node statements.

```
#let src = ``dot
digraph { a [shift="0.1,0", eval=legacy,
label=A]; }
```


```
#let n = nodes(parse(src.text).first()).first()
#n.shift
#n.statements
```


```

```
(x: 0.1, y: 0.0) (eval: "legacy", label:
"A", shift: "0.1,0")
```

Edge handling:

- `id=<n>` is consumed as the edge index. It is not preserved as a statement; drawing callbacks expose the resulting stable edge index as `eid`. Explicit edge indexes must be unique and in bounds. Without explicit `ids`, edge order is parser-internal, not DOT input order.

```
#let src = ``dot
digraph { a -> b [id=1, label=later]; b -> c
[id=0, label=first]; }
```


```
#let parsed-edges =
edges(parse(src.text).first())
#parsed-edges.map(e => e.edge)
#parsed-edges.map(e => e.statements.label)
#parsed-edges.map(e => e.statements.at("id",
```


```

```
default: none))
```

```
(0, 1) ("first", "later") (none, none)
```

- `dir=forward`, `dir=back`, and `dir=none` become edge orientations "default", "reversed", and "undirected". If omitted, directed DOT edges are "default" and undirected DOT edges are "undirected".

```
#let src = ``dot
digraph { a -> b [id=0]; b -> c [id=1,
dir=back]; c -> d [id=2, dir=none]; }
``

#edges(parse(src.text).first()).map(e =>
e.orientation)
```

```
("default", "reversed", "undirected")
```

- `source="..."` and `sink="..."` are consumed as endpoint statement values and removed from edge statements. On an external edge, use the attribute matching the real endpoint side in the DOT edge: `ext -> a` uses `sink=...`, while `a -> ext` uses `source=...`

```
#let src = ``dot
digraph { ext [style=invis]; ext -> a [id=0,
sink=in]; a -> ext [id=1, source=out]; }
``

#let endpoint-statement(endpoint) = if endpoint
== none { none } else { endpoint.at("statement",
default: none) }
#let parsed-edges =
edges(parse(src.text).first())
#parsed-edges.map(e => endpoint-
statement(e.source))
#parsed-edges.map(e => endpoint-
statement(e.sink))
#parsed-edges.map(e => e.statements.at("source",
default: none))
```

```
(none, "out") ("in", none) (none, none)
```

- `pos` is parsed as the edge control-point placement. `pin` is an explicit edge layout constraint. `shift`, `label-pos`, `label-angle`, and `bend` are parsed into edge geometry fields; except for `pin`, they also remain available as statements. Other edge attributes are preserved as edge statements.

```
#let src = ``dot
digraph { a -> b [id=0, pos="0,1!",
pin="x:@edge", shift="0.1,0", "label-
pos"="0,1.2", "label-angle"="0.3rad",
bend="0.4rad", particle=g]; }
``

#let e = edges(parse(src.text).first()).first()
#e.pos
#e.shift
#e.label-pos
#e.label-angle
#e.bend
#e.statements.particle
#e.statements.at("pin", default: none)
```

```
(x: 0.1, y: 0.0) 0.4 g
```

- Attributes copied from an invisible endpoint node are merged into the external edge statements unless their keys are `shape` or `label`.



```
#let src = ``dot
digraph { ext [style=invis, column=left,
shape=none, label=skip]; ext -> a [id=0]; }
```


```
#edges(parse(src.text).first()).first().statements
```


```

```
(column: "left")
```

Port and half-edge handling:

- DOT ports of the form node:port and node:port:compass are preserved on source/sink endpoint records. Numeric ports also assign explicit half-edge ids. Non-numeric ports are exposed only as port-label.

```
#let src = ``dot
digraph { a:left:e -> b:0:w [id=0]; }
```


```
#let e = edges(parse(src.text).first()).first()
#e.source.port-label
#e.sink.hedge
```


```

```
left 0
```

- Compass values are exposed as compass; valid DOT compass names include n, ne, e, se, s, sw, w, nw, c, and \_.

```
#let src = ``dot
digraph { a:n -> b:sw [id=0]; }
```


```
#let e = edges(parse(src.text).first()).first()
#e.source.compass
#e.sink.compass
```


```

```
n s
```

- Explicit half-edge ids from numeric ports must be unique and in bounds. They are retained for half-edge ordering and for join() with key: "id"; query and eval records expose the resulting half-edge index as hedge.

```
#let src = ``dot
digraph { a:1 -> b:0 [id=0]; }
```


```
#let e = edges(parse(src.text).first()).first()
#e.source.hedge
#e.sink.hedge
```


```

```
1 0
```

Placement fields:

- pos="x,y" gives a starting coordinate; pos="x,y!" pins both axes.

```
#let src = ``dot
digraph { a [id=0, pos="0,0"]; b [id=1,
pos="2,0!"]; }
```


```
#let parsed-nodes =
nodes(parse(src.text).first())
#parsed-nodes.map(n => n.pos)
#parsed-nodes.map(n => n.statements.at("pos-
mode", default: none))
```


```

```
((x: 0.0, y: 0.0), none) (none, none)
```

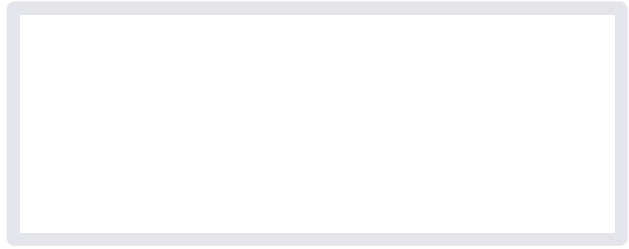
- pos="ref(node:<id>)+dx,dy!" and pos="ref(edge:<id>)+dx,dy!" reference explicit DOT node or edge ids, not names and not implicit parse order.

```
#let src = ``dot
digraph { a [id=0, pos="1,1!"]; b [id=1,
pos="ref(node:0)+2,0!"]; a -> b [id=0,
pos="ref(node:1)+0,1!"]; }
```


```
#let g = parse(src.text).first()
#nodes(g).at(1).pos
```

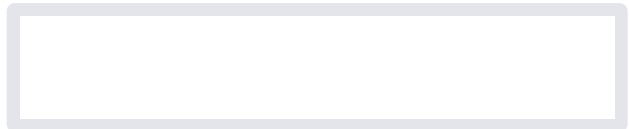

```

```
#edges(g).first().pos
```



- Axis form accepts x:<coord> and y:<coord> entries. ! applies to the individual axis, for example pos="x:2!,y:0".

```
#let src = ``dot
digraph { a [id=0, pos="x:2!,y:0"]; }
``
#nodes(parse(src.text).first()).first().pos
```



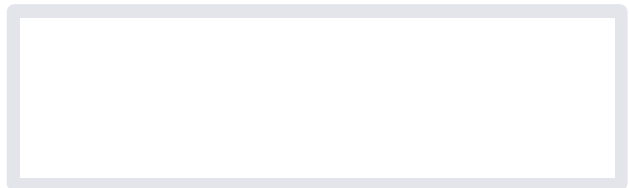
- Group coordinates use @name, @+name, or @-name in axis form and must be pinned with !, for example pos="x:@-left!,y:@row!".

```
#let src = ``dot
digraph { ext [style=invis]; ext -> a [id=0,
pos="x:@-left!,y:@row!"]; }
``
#edges(layout(parse(src.text).first(), layout-
algo: "tree")).first().pos
```

(x: -1.575, y: 1.0)

- pin accepts numeric point constraints, x:<coord>, y:<coord>, and grouped constraints such as x:@left, x:@+right, or @row.

```
#let src = ``dot
digraph { a -> b [id=0, pin="x:@+right"]; }
``
#edges(layout(parse(src.text).first(), layout-
algo: "tree")).first().statements.at("pin",
default: none)
```



## Parameters

```
parse(
 input: string bytes ,
 default-node-data: any ,
 eval-node-fields: string array ,
 default-edge-data: any ,
 eval-edge-fields: string array ,
 default-source-data: any ,
 eval-source-fields: string array ,
 default-sink-data: any ,
 eval-sink-fields: string array ,
 eval-graph-fields: string array ,
 eval-mode: string ,
 scope: dictionary
) -> array
```

**input** string or bytes

DOT source text containing one or more graph or digraph definitions.

### default-node-data any

Default data merged into every node data. Captured node data fields override it, but bare dot statements don't set data fields. To turn statements into data fields, use `eval-node-fields`.

```
#let a = ``dot
digraph {
 a -> b -> c -> d -> a
 a -> a
 b -> d
 c [particle="q"]
}
``

#let g = parse(a.text, default-node-data:
 (particle: "g")).at(0)
#nodes(g).map(n=>n.data.particle)
```

("g", "g", "g", "g")

Default: **none**

### eval-node-fields string or array

Node fields to evaluate into node data. The eval scope includes node statements plus node, name, and pos.

```
#let src = ``dot
digraph { a [label="#str(name)"]; }
``

#nodes(parse(src.text, eval-node-fields:
 "label").first()).first().data.label
```

a

Default: ()

### default-edge-data any

Default data merged into every edge data. Captured edge data fields override it, but bare dot statements don't set data fields. To turn statements into data fields, use `eval-edge-fields`.

Default: **none**

### eval-edge-fields string or array

Edge statement fields to evaluate into `graph.edges(g).at(i).data`. The eval scope includes edge statements plus edge, orientation, endpoint records, placement fields, and existing edge data. Drawing callbacks later expose the edge index as `eid`.

```
#let src = ``dot
digraph { a -> b [id=0,
 label="#orientation"]; }
``

#edges(parse(src.text, eval-edge-fields:
 "label").first()).first().data.label
```

default

Default: ()

**default-source-data** any

Default data merged into every source half-edge data. Captured source data fields override it.

Default: **none**

**eval-source-fields** string or array

Source half-edge fields to evaluate into `edge.source.data`. The eval scope includes source endpoint fields statement, port-label, compass, node, and hedge, plus the surrounding edge fields.

```
#let src = ``dot
digraph { a:left:e -> b [id=0, source="#port-
label"]; }
...
#edges(parse(src.text, eval-source-fields:
"statement").first()).first().source.data.statement
```

left

Default: ()

**default-sink-data** any

Default data merged into every sink half-edge data. Captured sink data fields override it.

Default: **none**

**eval-sink-fields** string or array

Sink half-edge fields to evaluate into `edge.sink.data`. The eval scope includes sink endpoint fields statement, port-label, compass, node, and hedge, plus the surrounding edge fields.

```
#let src = ``dot
digraph { a -> b:0:w [id=0,
sink="#str(hedge)"]; }
...
#edges(parse(src.text, eval-sink-fields:
"statement").first()).first().sink.data.statement
```

0

Default: ()

**eval-graph-fields** string or array

Graph statement fields to evaluate into `graph.info(g).data`. The eval scope includes graph statements and direct graph record fields.

```
#let src = ``dot
digraph {
 title = "Strong graph";
}
...
#info(parse(src.text, eval-graph-fields:
"title").first()).data.title
```

Strong graph

Default: ()

**eval-mode** `string`

Typst eval mode used for string field values.

Default: `"markup"`

**scope** `dictionary`

Additional Typst names available while evaluating field values.

Default: `(:)`

## node

Create a graph node item for `build()`.

A Typst label is the node name used by `source()`, `sink()`, and `pos()`. The optional numeric `id` fixes the resulting graph node index. Extra named arguments are captured as node data fields, so `node(<a>, label: [A], color: red)` stores `(label: [A], color: red)`. The default draw style uses `data.label` as the visible node label when present.

### Parameters

```
node(
 ..args: label,
 name: none label,
 id: none int,
 pos: none dictionary,
 shift: none string array dictionary,
 statements: dictionary
) -> array
```

**..args** `label`

Optional positional node name. Must be a Typst label when provided; extra named arguments become data fields.

**name** `none` or `label`

Typst node name for references.

Default: `none`

**id** `none` or `int`

Numeric graph node index. Must be unique and in bounds when provided.

Default: `none`

**pos** `none` or `dictionary`

Node placement.

Default: `none`

**shift** `none` or `string` or `array` or `dictionary`

Drawing shift stored as a statement.

Default: `none`

**statements** `dictionary`

Additional flat node statements. Used by DOT; values cannot nest.

Default: `(:)`

## source

Create a source half-edge endpoint.

node may be a node name like <a> or a numeric node index. name gives the half-edge a Typst name; id is a numeric half-edge order/index override. Extra named arguments are captured as source data fields.

### Parameters

```
source(
 node: label int,
 ..args: any,
 name: none label,
 id: none int,
 statement: none string,
 compass: none string
) -> dictionary
```

**node** `label` or `int`

Referenced node, either by Typst label name or numeric node index.

**..args** `any`

Extra named arguments become source data fields.

**name** `none` or `label`

Optional half-edge name used for later references.

Default: `none`

**id** `none` or `int`

Optional numeric half-edge index/order override.

Default: `none`

**statement** `none` or `string`

DOT-ish flat statement used for matching/joining dangling half edges.

Default: `none`

**compass** `none` or `string`

DOT compass point such as "n", "s", "e", or "w".

Default: `none`

## sink

Create a sink half-edge endpoint.

node may be a node name like <a> or a numeric node index. name gives the half-edge a Typst name; id is a numeric half-edge order/index override. Extra named arguments are captured as sink data fields.

### Parameters

```
sink(
 node: label int,
 ..args: any,
 name: none label,
 id: none int,
 statement: none string,
 compass: none string
) -> dictionary
```

**node** `label` or `int`

Referenced node, either by Typst label name or numeric node index.

**..args** `any`

Extra named arguments become sink data fields.

**name** `none` or `label`

Optional half-edge name used for later references.

Default: `none`

**id** `none` or `int`

Optional numeric half-edge index/order override.

Default: `none`

**statement** `none` or `string`

DOT-ish flat statement used for matching/joining dangling half edges.

Default: `none`

**compass** `none` or `string`

DOT compass point such as "n", "s", "e", or "w".

Default: `none`

## edge

Create a graph edge item for `build()`.

Positional arguments may contain one `source()`, one `sink()`, and optionally one Typst label used as the edge name. The numeric id chooses the edge order. Extra named arguments are captured as edge data fields, so `edge(source(<a>), sink(<b>), particle: "g")` stores `(particle: "g")`. The default draw style uses `data.label` as the visible edge label when present.

### Parameters

```
edge(
 ..args: any,
 name: none label,
 id: none int,
 orientation: string,
 pos: none dictionary,
 shift: none string array dictionary,
 label-pos: none string array dictionary,
 label-angle: none int float string,
 bend: none int float string,
 statements: dictionary
) -> array
```

**..args** any

Source/sink half-edges and optional edge name; extra named arguments become data fields.

**name** none or label

Typst edge name.

Default: none

**id** none or int

Numeric edge order/index override.

Default: none

**orientation** string

Edge orientation: "default", "reversed", or "undirected".

Default: "default"

**pos** none or dictionary

Edge placement.

Default: none

**shift** none or string or array or dictionary

Drawing shift stored as a statement.

Default: none



**label-pos** `none` or `string` or `array` or `dictionary`

Edge label position stored as a statement.

Default: `none`

**label-angle** `none` or `int` or `float` or `string`

Edge label angle stored as a statement.

Default: `none`

**bend** `none` or `int` or `float` or `string`

Edge bend stored as a statement.

Default: `none`

**statements** `dictionary`

Additional flat edge statements. Used by DOT; values cannot nest.

Default: `( : )`

## group

Create a grouped placement coordinate.

### Parameters

```
group(
 name: string int bool ,
 side: none string
) -> dictionary
```

**name** `string` or `int` or `bool`

Group identifier shared by positions constrained to the same coordinate.

**side** `none` or `string`

Optional sign constraint: "+", "-", "positive", or "negative".

Default: `none`

## pos

Create a first-class graph placement.

## Parameters

```
pos(
 x: none int float dictionary ,
 y: none int float dictionary ,
 ref: none label int ,
 dx: none int float ,
 dy: none int float ,
 mode: string
) -> dictionary
```

**x** **none** or **int** or **float** or **dictionary**

Absolute or grouped x coordinate.

Default: **none**

**y** **none** or **int** or **float** or **dictionary**

Absolute or grouped y coordinate.

Default: **none**

**ref** **none** or **label** or **int**

Node reference for relative placement, by name or numeric index.

Default: **none**

**dx** **none** or **int** or **float**

Relative x offset from ref.

Default: **none**

**dy** **none** or **int** or **float**

Relative y offset from ref.

Default: **none**

**mode** **string**

Placement mode: "pin" constrains layout, "start" only seeds it.

Default: **"pin"**

## map

Map graph metadata to new native data.

## Parameters

```
map(
 graph_: dictionary ,
 graph: none or function ,
 node: none or function ,
 edge: none or function ,
 source: none or function ,
 sink: none or function
) -> dictionary
```

**graph\_** dictionary

Graph object to transform.

**graph** none or function

Callback for graph metadata records.

Default: none

**node** none or function

Callback for node records.

Default: none

**edge** none or function

Callback for edge records.

Default: none

**source** none or function

Callback for source half-edge records.

Default: none

**sink** none or function

Callback for sink half-edge records.

Default: none

## eval-fields

Evaluate selected fields into native data entries.

## Parameters

```
eval-fields(
 graph_: dictionary ,
 eval-graph-fields: string array ,
 eval-node-fields: string array ,
 eval-edge-fields: string array ,
 eval-source-fields: string array ,
 eval-sink-fields: string array ,
 eval-mode: string ,
 scope: dictionary
) -> dictionary
```

**graph\_** dictionary

Graph object whose selected statement fields should be evaluated.

**eval-graph-fields** string or array

Graph statement fields to evaluate into `graph.info(g).data`.

Default: ()

**eval-node-fields** string or array

Node statement fields to evaluate into node data.

Default: ()

**eval-edge-fields** string or array

Edge statement fields to evaluate into edge data.

Default: ()

**eval-source-fields** string or array

Source half-edge fields to evaluate into source data.

Default: ()

**eval-sink-fields** string or array

Sink half-edge fields to evaluate into sink data.

Default: ()

**eval-mode** string

Typst eval mode used for string field values.

Default: "markup"

**scope**    dictionary

Additional Typst names available while evaluating field values.

Default: ( : )

## info

Return graph metadata.

### Parameters

`info`(graph: dictionary) -> dictionary

**graph**    dictionary

Graph object returned by `build()`, `parse()`, `layout`, or another graph API.

## dot

Serialize a graph object to DOT.

### Parameters

`dot`(graph: dictionary) -> string

**graph**    dictionary

Graph object to serialize.

## nodes

Return node records, optionally filtered by a subgraph object.

Node name values are Typst labels when present.

### Parameters

```
nodes(
 graph: dictionary ,
 subgraph: none bytes
) -> array
```

**graph**    dictionary

Graph object to inspect.

**subgraph**    none or bytes

Optional subgraph filter; only nodes incident to selected half edges are returned.

Default: none

## edges

Return edge records, optionally filtered by a subgraph object.

Edge name values are Typst labels when present.

### Parameters

```
edges(
 graph: dictionary,
 subgraph: none or bytes
) -> array
```

**graph** dictionary

Graph object to inspect.

**subgraph** none or bytes

Optional subgraph filter; only selected edges/half-edges are returned.

Default: none

## node-data

Return one named node's native data.

### Parameters

```
node-data(
 graph_: dictionary,
 name: label string
) -> any
```

**graph\_** dictionary

Graph object to inspect.

**name** label or string

Node name as a Typst label or its string form.

## edge-data

Return one named edge's native data.

### Parameters

```
edge-data(
 graph_: dictionary,
 name: label string
) -> any
```

**graph\_** dictionary

Graph object to inspect.

**name**    label or string

Edge name as a Typst label or its string form.

### update-node-data

Update one named node's native data.

#### Parameters

```
update-node-data(
 graph_: dictionary ,
 name: label string ,
 update: any function
) -> dictionary
```

**graph\_**    dictionary

Graph object to update.

**name**    label or string

Node name as a Typst label or its string form.

**update**    any or function

Replacement data or (data, node) => new-data callback.

### update-edge-data

Update one named edge's native data.

#### Parameters

```
update-edge-data(
 graph_: dictionary ,
 name: label string ,
 update: any function
) -> dictionary
```

**graph\_**    dictionary

Graph object to update.

**name**    label or string

Edge name as a Typst label or its string form.

**update**    any or function

Replacement data or (data, edge) => new-data callback.

## join

Join two graphs by matching dangling half-edge statements or ids on key.

### Parameters

```
join(
 left: dictionary,
 right: dictionary,
 key: string
) -> dictionary
```

**left** dictionary

Left graph object.

**right** dictionary

Right graph object.

**key** string

Dangling half-edge match key: "statement", "compass", or "id".

Default: "statement"

## cycles

Return subgraph objects for the graph's cycle basis.

### Parameters

```
cycles(graph: dictionary) -> array
```

**graph** dictionary

Graph object to analyze.

## forests

Return subgraph objects for the graph's spanning forests.

### Parameters

```
forests(graph: dictionary) -> array
```

**graph** dictionary

Graph object to analyze.

## layout

- layout()

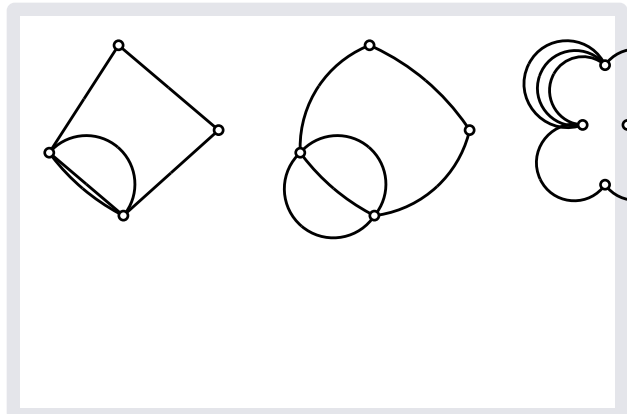
### layout

Apply the linnest layout pass to a graph object.



This is intentionally a second step: construct or parse a graph first, then call layout. Set layout-algo to "force" for deterministic force integration, "anneal" for simulated annealing, "tree" for a traversal tree placement, or "dot" for a layered directed placement.

```
#let g = graph.parse("digraph partial { a -> b; a
-> b; a:s -> b:s; b:s -> c:s; c:s -> d:s; d:s ->
a:s }").at(0)
#let south = subgraph.compass(g, "s")
#let gf = layout(layout(g, layout-algo:
"force"), layout-algo: "tree", layout-nodes:
"fixed", subgraph:south)
#let ga = layout(g, layout-algo: "force")
#let gt = layout(layout(g, layout-algo:
"tree", layout-roots: (2)), layout-algo:
"anneal", layout-nodes: "fixed", gamma-
ee:0.1, gamma-ev:.75, beta:5, length-scale:0.4)
#grid(columns: 3, gutter: 2cm, draw(gf),
draw(ga), draw(gt))
```



```
#let g = graph.parse("digraph partial { a -> b;
b -> c; c -> d; d -> a }").at(0)
#let tree = graph.forests(g).at(0)
#let g = layout(g, layout-algo: "tree",
subgraph: tree)
#graph.edges(g).map(edge => edge.pos)
```

```
(
(x: 0.0, y: 1.05),
(x: 0.0, y: 3.1500000000000004),
(x: 0.0, y: 5.25),
(x: 0.0, y: 3.1500000000000004),
)
```

## Parameters

```
layout(
 graph: dictionary ,
 subgraph: none bytes ,
 viewport-w: float ,
 viewport-h: float ,
 tree-dx: float ,
 tree-dy: float ,
 steps: int ,
 seed: int ,
 step: float ,
 step-shrink: float ,
 cool: float ,
 accept-floor: float ,
 early-tol: float ,
 temp: float ,
 delta: float ,
 beta: float ,
 k-spring: float ,
 g-center: float ,
 epochs: int ,
 crossing-penalty: float ,
 gamma-dangling: float ,
 gamma-ee: float ,
 directional-force: float ,
 label-length-scale: float ,
 label-spring: float ,
 label-charge: float ,
 label-steps: int ,
 label-step: float ,
 label-early-tol: float ,
 label-max-delta-scale: float ,
 gamma-ev: float ,
 eps: float ,
 incremental-energy: bool ,
 layout-algo: string ,
 layout-nodes: string ,
 layout-roots: array ,
 z-spring: float ,
 z-spring-growth: float ,
 length-scale: float
) -> dictionary
```

**graph** dictionary

Graph object returned by `graph.build` or `graph.parse`.

**subgraph** none or bytes

Optional subgraph object to lay out. With "tree" and "dot", other edges are drawn from the resulting node positions as straight lines. With "force" and "anneal", nodes and edges outside the subgraph are fixed boundary points during optimization.

Default: none

**viewport-w** float

Width of the layout viewport used to derive the natural spring length. Applies to both "force" and "anneal".

Default: 10.0

**viewport-h** float

Height of the layout viewport used to derive the natural spring length. Applies to both "force" and "anneal".

Default: 10.0

**tree-dx** float

Horizontal spacing multiplier for the initial traversal-tree placement. Applies before both "force" and "anneal".

Default: 0.9

**tree-dy** float

Vertical spacing multiplier for the initial traversal-tree placement. Applies before both "force" and "anneal".

Default: 1.2

**steps** int

Iterations per epoch. In "force" mode this is the number of force integration steps; in "anneal" mode this is the number of proposals per temperature epoch.

Default: `int(sys.inputs.at("steps", default: "30"))`

**seed** int

Seed for deterministic initialization, force-mode jitter, and annealing proposals. Applies to both modes.

Default: `int(sys.inputs.at("seed", default: "2"))`

**step** float

Initial movement scale. "force" multiplies computed forces by this value; "anneal" uses it as the proposal step size.

Default: 0.81

**step-shrink** float

Anneal-only step shrink factor, applied when an epoch's acceptance ratio falls below `accept-floor`.

Default: 0.21

**cool** float

Cooling factor applied once per epoch. "anneal" cools temp; "force" shrinks step.

Default: 0.85

**accept-floor** float

Anneal-only acceptance-ratio threshold below which step is shrunk by step-shrink.

Default: 0.15

**early-tol** float

Force-only early stop threshold for maximum movement in one step. The annealing schedule stores this value but does not currently use it for stopping.

Default: 1e-6

**temp** float

Anneal-only initial temperature used in the Metropolis acceptance test.

Default: 0.3

**delta** float

Force-mode maximum movement clamp per point and per step.

Default: 0.4

**beta** float

Base repulsion strength for vertex-vertex interactions. Also scales gamma-ev, gamma-ee, gamma-dangling, and g-center. Applies to both modes through the shared spring energy.

Default: 50.0

**k-spring** float

Spring stiffness for node-to-edge incidence lengths. Applies to both modes.

Default: 11.0

**g-center** float

Centering strength relative to beta. Applies to both modes.

Default: 0.002

**epochs** int

Number of epochs. Both modes run up to steps iterations inside each epoch.

Default: 30

**crossing-penalty** float

Anneal-only fixed energy penalty per detected edge crossing. The direct force integrator does not currently add a crossing force.

Default: 30.0

**gamma-dangling** float

Repulsion for dangling half edges, relative to beta. Applies to both modes through the shared spring energy.

Default: 5.0

**gamma-ee** float

Local edge-edge repulsion, relative to beta. Applies to both modes.

Default: 0.1

**directional-force** float

Bias that pushes points in directions implied by pin/port constraints. Applies to both modes.

Default: 5.0

**label-length-scale** float

Edge-label target offset as a multiple of the graph spring length. Label layout runs after both graph layout modes.

Default: 0.6

**label-spring** float

Spring strength pulling each label toward its target offset. Label layout runs after both modes.

Default: 23.0

**label-charge** float

Repulsion strength between labels and graph points, scaled by spring length squared. Label layout runs after both modes.

Default: 3.0

**label-steps** int

Maximum number of post-layout label relaxation steps. Set to 0 to skip label placement. Applies after both modes.

Default: 20

**label-step** float

Label relaxation step size. Applies after both modes.

Default: 0.15

**label-early-tol** float

Label relaxation early stop threshold. Applies after both modes.

Default: 1e-3

**label-max-delta-scale** float

Label movement clamp as a multiple of spring length. Applies after both modes.

Default: 0.5

**gamma-ev** float

Edge-vertex repulsion, relative to beta. Applies to both modes.

Default: 0.01

**eps** float

Softening epsilon used in inverse-square force/energy terms. Applies to both modes.

Default: 1e-4

**incremental-energy** bool

Whether annealing updates cached energy by local deltas. This is anneal-only; force mode computes direct forces instead of energies.

Default: true

**layout-algo** string

Layout algorithm. Use "force" for direct force integration, "anneal" for simulated annealing against the spring energy, "tree" for a traversal-tree placement, or "dot" for a layered directed placement.

Default: "force"

**layout-nodes** string

Node movement policy. "layout" lets the layout algorithm move nodes. "fixed" keeps every node at its current position for this layout pass and only moves edge control points. With subgraph, only edges in the subgraph are moved; other edge control points stay at their current positions.

Default: "layout"

**layout-roots**    array

Ordered node indices used as preferred roots for "tree" and "dot". Roots outside the selected node set are ignored. Remaining components are laid out afterward in graph order.

Default: ()

**z-spring**    float

Force-only spring pulling temporary z coordinates back toward the layout plane. Use with z-spring-growth to help separate overlapping points during integration.

Default: 0.05

**z-spring-growth**    float

Force-only per-epoch multiplier for z-spring.

Default: 1.3

**length-scale**    float

Natural spring-length multiplier. This scales the graph's preferred edge length and the repulsive coefficients derived from it. Applies to both modes.

Default: 0.35

## physics

- stroke-style()
- source-stroke()
- sink-stroke()
- particle-name()
- edge-entry()
- text-value()
- eval-expression()
- label-content()
- style-dict()
- source-style()
- sink-style()
- momentum-value()
- edge-index()
- dangling-half-edge-index()
- edge-label()
- style()

## Variables

- massive
- massless
- dashed
- dotted
- fermion-arrow-mark
- fermion-flow
- wave
- coil
- zigzag
- default-edge

- default-map
- momentum-arrow-defaults

### stroke-style

Build a rounded CeTZ stroke dictionary accepted by `linnest.draw`.

#### Parameters

```
stroke-style(
 c,
 thickness,
 dash
) -> dictionary
```

### source-stroke

Source-half stroke helper.

#### Parameters

```
source-stroke(
 c,
 thickness,
 dash
) -> dictionary
```

### sink-stroke

Sink-half stroke helper. The default lightening makes the two halves encode the graph's source/sink split without requiring arrowheads.

#### Parameters

```
sink-stroke(
 c,
 thickness,
 dash,
 lighten
) -> dictionary
```

### particle-name

Return an edge's particle name, stripping the quotes often present in DOT statement values.

#### Parameters

```
particle-name(edge) -> none string
```

### edge-entry

Return the style-map entry for an edge.

#### Parameters

```
edge-entry(
 edge,
 map,
 default
) -> dictionary
```



### text-value

Convert DOT-ish values to plain text.

#### Parameters

```
text-value(value) -> string
```

### eval-expression

Evaluate a string-valued style expression in the edge scope.

#### Parameters

```
eval-expression(
 value,
 edge,
 map,
 scope
) -> any
```

### label-content

Convert a style label value to content. In mode: "eval", string values are evaluated in the edge scope.

#### Parameters

```
label-content(
 value,
 edge,
 mode,
 map,
 scope
) -> none content
```

### style-dict

Convert a style value to a dictionary. In mode: "eval", string values are evaluated in the edge scope.

#### Parameters

```
style-dict(
 value,
 edge,
 mode,
 map,
 scope
) -> dictionary
```

### source-style

Style callback for the source half edge.

momentum-arrows: true adds one centered black CeTZ-mark decoration while the main edge remains drawn with its normal particle style.

## Parameters

```
source-style(
 edge,
 map,
 default,
 typst-fields,
 scope,
 orientation-split,
 momentum-arrows,
 momentum-arrow-offset,
 momentum-arrow-length,
 momentum-arrow-ratio,
 momentum-arrow-stroke,
 momentum-arrow-mark
) -> dictionary array
```

## sink-style

Style callback for the sink half edge.

`momentum-arrows: true` adds one centered black CeTZ-mark decoration toward the sink node, so momentum arrows always flow from source to sink independently of `edge.orientation`.

## Parameters

```
sink-style(
 edge,
 map,
 default,
 typst-fields,
 scope,
 orientation-split,
 momentum-arrows,
 momentum-arrow-offset,
 momentum-arrow-length,
 momentum-arrow-ratio,
 momentum-arrow-stroke,
 momentum-arrow-mark
) -> dictionary array
```

## momentum-value

Return the momentum field used by optional edge labels.

## Parameters

```
momentum-value(
 edge,
 fields
) -> none any
```

## edge-index

Return the edge index used by optional edge labels. The DOT `id` statement wins over the renderer-local `eid`.

## Parameters

```
edge-index(
 edge,
 fields
) -> none any
```

## dangling-half-edge-index

Return the exposed half-edge index for dangling edges only.

## Parameters

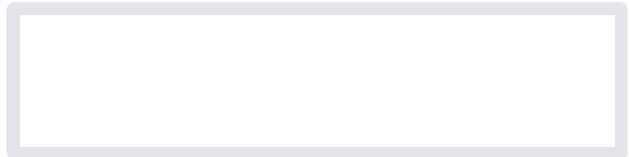
```
dangling-half-edge-index(edge) -> none any
```

## edge-label

Edge-label callback.

By default this preserves data display-label or label, then explicit display-label, label, and particle-map labels. Set any show-\* option to build a label from selected metadata instead:

```
#let callbacks = physics.style(
 momentum-arrows: true,
 show-edge-index: true,
 show-particle: true,
)
```



## Parameters

```
edge-label(
 edge,
 map,
 default,
 typst-fields,
 scope,
 show-momentum,
 show-edge-index,
 show-half-edge-index,
 show-particle,
 momentum-fields,
 edge-index-fields,
 momentum-prefix,
 edge-index-prefix,
 half-edge-index-prefix,
 particle-prefix,
 label-separator,
],
 label-size,
 label-fill
) -> none content
```

## style

Bundle source-style, sink-style, and edge-label callbacks with shared options for linnest.draw.

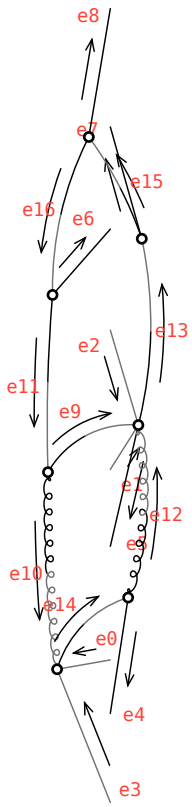
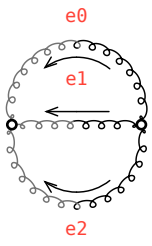
```
#let g = build({
 node(<a>, pos: pos(y:0))
```

```

 node(,pos:pos(y:0))
 edge(source(<a>), <g-edge>, sink(),
particle: "g")
 edge(source(<a>), <g-edge2>, sink(),
particle: "g")
 edge(source(<a>), <g-edge3>, sink(),
particle: "g")
 },
 name: "physics",
)
#let a = ``dot
digraph {
0 [dod="-100" num="1"];
1 [dod="-100" num="1"];
2 [dod="-100" num="1"];
3 [dod="-100" num="1"];
4 [dod="-100" num="1"];
5 [dod="-100" num="1"];
6 [dod="-100" num="1"];
exte0 [style=invis];
exte0 -> 2:0 [id=0 dod="-100"
lmb_rep="P(0,a___)" num="1" particle="d"
pin="x:@-left"];
exte1 [style=invis];
exte1 -> 1:1 [id=1 dir=back dod="-100"
lmb_rep="P(1,a___)" num="1" particle="d-"
pin="x:@-left"];
exte2 [style=invis];
exte2 -> 1:2 [id=2 dir=none dod="-100"
lmb_rep="P(2,a___)" mass="0" num="1"
particle="H" pin="x:@-left"];
exte3 [style=invis];
exte3 -> 2:3 [id=3 dir=none dod="-100"
is_dummy="true" lmb_rep="0" mass="0" num="1"
particle="H" pin="x:@-left"];
exte4 [style=invis];
6:4 -> exte4 [id=4 dir=none dod="-100"
lmb_rep="P(3,a___)" mass="0" num="1"
particle="H" pin="x:@+right"];
exte5 [style=invis];
1:5 -> exte5 [id=5 dir=none dod="-100"
is_dummy="true" lmb_rep="0" mass="0" num="1"
particle="H" pin="x:@+right"];
exte6 [style=invis];
5:6 -> exte6 [id=6 dir=none dod="-100"
lmb_rep="P(4,a___)" num="1" particle="H"
pin="x:@+right"];
exte7 [style=invis];
4:7 -> exte7 [id=7 dir=none dod="-100"
lmb_rep="P(5,a___)" num="1" particle="H"
pin="x:@+right"];
exte8 [style=invis];
3:8 -> exte8 [id=8 dir=none dod="-100"
lmb_rep="-1*P(3,a___)+-1*P(4,a___)+-1*P(5,a___)+P(0,a___)+P(1,a___)+P(2,a___)"
num="1" particle="H" pin="x:@+right"];
0:9 -> 1:10 [id=9 dod="-100" lmb_id="1"
lmb_rep="K(1,a___)" num="1" particle="t"];
0:11 -> 2:12 [id=10 dir=none dod="-100"
lmb_rep="-1*P(0,a___)+K(0,a___)" num="1"
particle="g"];
5:13 -> 0:14 [id=11 dod="-100"
lmb_rep="-1*P(0,a___)+K(0,a___)+K(1,a___)"
num="1" particle="t"];
6:24 -> 1:18 [id=12 dir=none dod="-100"
lmb_rep="-1*P(3,a___)+K(0,a___)"

```

[illegible]



## Parameters

```
style(
 map,
 default,
 typst-fields,
 scope,
 orientation-split,
 momentum-arrows,
 momentum-arrow-offset,
 momentum-arrow-length,
 momentum-arrow-ratio,
 momentum-arrow-stroke,
 momentum-arrow-mark,
 show-momentum,
 show-edge-index,
 show-half-edge-index,
 show-particle,
 momentum-fields,
 edge-index-fields,
 momentum-prefix,
 edge-index-prefix,
 half-edge-index-prefix,
 particle-prefix,
 label-separator,
],
 label-size,
 label-fill
) -> dictionary
```

**massive**    length

Conventional massive-particle stroke thickness.

**massless**    length

Conventional massless-particle stroke thickness.

**dashed**    array

Dash pattern used for scalar-style lines.

**dotted**    string

Dotted dash style used for ghost-style lines.

**fermion-arrow-mark**    dictionary

CeTZ marker used for fermion particle-flow arrows on the main edge.

### **fermion-flow** dictionary

Mark an edge-map entry as a fermion so the main edge receives one particle-flow arrow that follows `edge.orientation`. This is separate from optional momentum arrows.

### **wave** dictionary

Photon-style wave pattern.

### **coil** dictionary

Gluon-style coil pattern.

### **zigzag** dictionary

Weak-boson-style zigzag pattern.

### **default-edge** dictionary

Fallback style entry used when an edge has no known particle type.

### **default-map** dictionary

Small built-in particle map for standalone physics diagrams. GammaLoop generated `edge-style.typ` files pass their model-specific map to `style`.

### **momentum-arrow-defaults** dictionary

Default style for source-to-sink momentum arrow layers.

### **draw**

- `edge-halves()`
- `to-cetz-edge-halves()`
- `draw()`

### **edge-halves**

Split a laid-out graph edge into source and sink half-edge paths.

The returned dictionary has `source`, `sink`, and `curve`. The split point is the edge layout point, so the two half-edges join smoothly there.



## Parameters

```
edge-halves(
 edge: dictionary ,
 nodes: array ,
 omega: float ,
 source-outset: int float ,
 sink-outset: int float ,
 accuracy: float
) -> dictionary
```

**edge** dictionary

Edge record returned by `graph.edges(layout(g))`.

**nodes** array

Node records returned by `graph.nodes(layout(g))`.

**omega** float

Hobby curl used for the source-to-edge and edge-to-sink curves.

Default: 1.0

**source-outset** int or float

Arc-length trim applied at the source node side.

Default: 0

**sink-outset** int or float

Arc-length trim applied at the sink node side.

Default: 0

**accuracy** float

Arc-length accuracy used while trimming.

Default: 0.001

## to-cetz-edge-halves

Draw the two halves of a laid-out graph edge through CeTZ.

`source-style` applies from the source node to the edge layout point, and `sink-style` applies from the edge layout point to the sink node.

## Parameters

```
to-cetz-edge-halves(
 edge: dictionary,
 nodes: array,
 unit: int float length ratio,
 omega: float,
 source-outset: int float,
 sink-outset: int float,
 accuracy: float,
 source-style: dictionary,
 sink-style: dictionary
) -> content
```

**edge** dictionary

Edge record returned by `graph.edges(layout(g))`.

**nodes** array

Node records returned by `graph.nodes(layout(g))`.

**unit** int or float or length or ratio

Coordinate length for one graph-layout unit. Numbers are interpreted as em.

Default: 1

**omega** float

Hobby curl used for the source-to-edge and edge-to-sink curves.

Default: 1.0

**source-outset** int or float

Arc-length trim applied at the source node side.

Default: 0

**sink-outset** int or float

Arc-length trim applied at the sink node side.

Default: 0

**accuracy** float

Arc-length accuracy used while trimming.

Default: 0.001

**source-style** dictionary

CeTZ style for the source half edge.

Default: (:)

**sink-style**    dictionary

CeTZ style for the sink half edge.

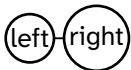
Default: ( : )

## draw

Draw a graph object with CeTZ.

The graph must already have positions, either from `layout` or from explicit `pos` values passed to `graph` node or edge items. Paired edges without an explicit edge position use the midpoint of their endpoint nodes. Node and edge Typst style dictionaries or callbacks are evaluated and forwarded to CeTZ.

```
#let positioned = graph.build({
 graph.node(<left>, label: [left], pos: graph.pos(x: 0, y: 0))
 graph.node(<right>, label: [right], pos: graph.pos(ref: <left>, dx: 2.5, dy: 0))
 graph.edge(graph.source(<left>), graph.sink(<right>))
 graph.edge(graph.source(<right>), <right-out>, pos: graph.pos(ref: <right>, dx: 0.9, dy: 0.7))
},
 name: "positioned",
)
#draw(positioned, source-style: (stroke: black + 0.7pt), sink-style: (stroke: black + 0.7pt))
```

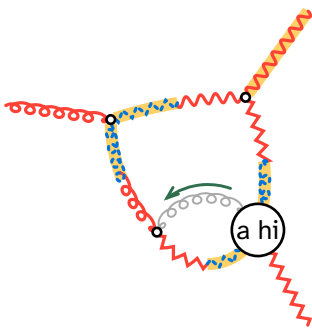


```
#let parallel-base-edge = 0
#let source-patterns = (
 "coil",
 "zigzag",
 "coil",
 "wave",
 "zigzag",
 "coil",
 "wave",
 "zigzag",
)
#let sink-patterns = (
 "coil",
 "zigzag",
 "coil",
 "zigzag",
 "coil",
 "wave",
 "coil",
 "wave",
)
#let parallel-layer(edge, mark: none) = if edge.eid == parallel-base-edge {
 (
 offset: -0.5,
 length: 1.5,
 ratio: 0.5,
 resolve-length: "min",
 stroke: (paint: rgb("#2f6f4e"), thickness: 1.1pt, cap: "round"),
 mark: mark,
)
} else { none }
#let stack(base, layer) = if layer == none { base } else { (base, layer) }
```

```

#let source-stroke(edge) = if edge.eid == parallel-base-edge {
 (paint: gray, thickness: 0.75pt, cap: "round")
} else {
 (paint: red, thickness: 1.2pt, cap: "round")
}
#let sink-stroke(edge) = if edge.eid == parallel-base-edge {
 (paint: gray, thickness: 0.75pt, cap: "round")
} else {
 (paint: blue, thickness: 1.2pt, cap: "round", dash: "dotted")
}
#let source-style(edge) = {
 let base = (
 stroke: source-stroke(edge),
 pattern: source-patterns.at(edge.eid),
 pattern-amplitude: 0.18,
 pattern-wavelength: 0.55,
 pattern-coil-longitudinal-scale: 1.6,
)
 stack(base, parallel-layer(edge))
}
#let sink-style(edge) = {
 let base = (
 stroke: sink-stroke(edge),
 pattern: sink-patterns.at(edge.eid),
 pattern-amplitude: 0.18,
 pattern-wavelength: 0.55,
 pattern-coil-longitudinal-scale: 1.6,
)
 stack(base, parallel-layer(edge, mark: (end: (symbol: "straight"), scale: 0.75)))
}
#let g = graph.build({
 graph.node(<a>, label: [a hi])
 graph.node(<c>)
 graph.node(<d>)
 graph.node(<e>)
 graph.edge(graph.source(<a>), <ac>, graph.sink(<c>), pos: graph.pos(x: 0, y: 0.75, mode: "pin"))
 graph.edge(graph.source(<c>), <ca>, graph.sink(<a>), compass: "e")
 graph.edge(graph.source(<c>), <cd>, graph.sink(<d>), compass: "e")
 graph.edge(graph.source(<e>), <ed>, graph.sink(<d>), compass: "e")
 graph.edge(graph.source(<e>), <ea>, graph.sink(<a>), compass: "e")
 graph.edge(graph.source(<d>), <d-out>)
 graph.edge(graph.source(<e>, compass: "e"), <e-out>)
 graph.edge(graph.source(<a>), <a-out>)
},
 name: "demo",
)
#let layed-out = layout(g)
#let east = subgraph.compass(layed-out, "e")
#draw(layed-out, subgraph: east, source-style: source-style, sink-style: sink-style)

```



```

#let p = graph.build({
 graph.node(<pa>, label: [a], pos: graph.pos(y: 0, mode: "pin"))

```

```

graph.node(<pc>, label: [c], pos: graph.pos(y: 0, mode: "pin"))
graph.node(<pc1>, label: [c], pos: graph.pos(y: 0, mode: "pin"))
graph.node(<pc2>, label: [c], pos: graph.pos(y: 0, mode: "pin"))
graph.edge(graph.source(<pa>), <e0>, graph.sink(<pc>))
graph.edge(graph.source(<pc2>), <e1>, graph.sink(<pc1>))
},
name: "parallel demo",
)
#let parallel-edge-style(edge) = (
 offset: -0.5,
 length: 1.4,
 ratio: 0.5,
 resolve-length: "min",
 stroke: (paint: rgb("#2f6f4e"), thickness: 1.1pt, cap: "round"),
)

#let focused-base-style(edge) = (
 stroke: (paint: gray, thickness: 0.7pt, cap: "round"),
 pattern: "coil",
 pattern-amplitude: 0.14,
 pattern-wavelength: 0.45,
 pattern-coil-longitudinal-scale: 1.5,
)
#let focused-source-style(edge) = (focused-base-style(edge), parallel-edge-style(edge))
#let focused-sink-style(edge) = (focused-base-style(edge), parallel-edge-style(edge) + (
 mark: (end: ">"),
))
#draw(layout(p, g-center: 0.004, label-steps: 0,), unit: 1.25, source-style: focused-source-style, sink-
style: focused-sink-style)

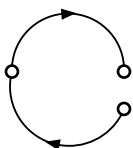
```



```

#let g = graph.build({
 graph.node(<a>, pos: graph.pos(x: 0, y: 0, mode: "pin"))
 graph.node(<c>, pos: graph.pos(x: 3, y: 0, mode: "pin"))
 graph.node(<d>, pos: graph.pos(x: 3, y: -1, mode: "pin"))
 graph.edge(graph.source(<a>), <ac>, graph.sink(<c>), orientation: "default")
 graph.edge(graph.source(<a>), <ad>, graph.sink(<d>), orientation: "reversed")
},
name: "oriented marks",
)
#let arrow = (
 end: (symbol: ">", fill: black, anchor: "center", shorten-to: auto),
 scale: 0.75,
)
#let oriented-arrow = (
 stroke: black + 0.7pt,
 mark: arrow,
 mark-position: "center-if-dangling",
 mark-orientation: "edge",
)
#draw(layout(g), unit: 1.4, source-style: oriented-arrow, sink-style: oriented-arrow)

```



## Parameters

```
draw(
 graph: bytes ,
 scope: dictionary ,
 unit: int float length ratio ,
 title: none auto content string ,
 subgraph: none bytes ,
 debug: bool int ,
 node-radius: auto int float array ,
 node-min-radius: int float ,
 node-label-padding: int float ,
 node-fill: any ,
 node-stroke: any ,
 node-outset: auto int float ,
 node-label-style: dictionary ,
 node-style: dictionary function none ,
 node-label: auto content string function none ,
 edge-stroke: any ,
 edge-offset: int float ,
 edge-length: none int float ,
 edge-ratio: none int float ,
 edge-resolve-length: string function ,
 edge-accuracy: float ,
 edge-optimize: bool ,
 source-style: dictionary array function none ,
 sink-style: dictionary array function none ,
 edge-label: content string function none ,
 edge-omega: float ,
 edge-trim-accuracy: float ,
 padding: none int float array dictionary ,
 debug-edge-radius: int float ,
 debug-edge-fill: any ,
 debug-edge-stroke: any ,
 debug-edge-label-fill: any ,
 subgraph-edge-style: dictionary ,
 subgraph-edge-underlay: bool
) -> content
```

**graph** bytes

Graph object with positions from layout or explicit graph API pos fields.

**scope** dictionary

Additional values merged into node and edge callback dictionaries.

Default: ( : )

**unit** int or float or length or ratio

Coordinate length for one graph-layout unit. Numbers are interpreted as em.

Default: 1

**title** `none` or `auto` or `content` or `string`

Optional title displayed above the diagram. Use auto for the graph name.

Default: `none`

**subgraph** `none` or `bytes`

Optional subgraph whose half-edges are shaded.

Default: `none`

**debug** `bool` or `int`

Debug level. 1 enables CeTZ canvas debug; 2 also marks edge positions.

Default: `false`

**node-radius** `auto` or `int` or `float` or `array`

Default CeTZ node radius. Use auto to fit the node label.

Default: `auto`

**node-min-radius** `int` or `float`

Minimum radius used when node-radius is auto.

Default: `0.16`

**node-label-padding** `int` or `float`

Extra canvas-unit padding around labels when node-radius is auto.

Default: `0.08`

**node-fill** `any`

Default CeTZ node fill.

Default: white

**node-stroke** `any`

Default CeTZ node stroke.

Default: black

**node-outset** `auto` or `int` or `float`

Edge clearance from node centers. auto uses each node circle radius. Increase this when labels extend beyond the circle.

Default: `auto`

**node-label-style** dictionary

Default node-label style forwarded to `cetz.draw.content`.

Default: `(:)`

**node-style** dictionary or function or none

Node style dictionary or callback. A callback receives node data.

Default: `(:)`

**node-label** auto or content or string or function or none

Node label content or callback. auto uses the node name.

Default: `auto`

**edge-stroke** any

Default CeTZ edge stroke.

Default: `0.1em`

**edge-offset** int or float

Default normal offset for edge paths. Applied to the base edge geometry before patterns; node outsets then trim the shifted path.

Default: `0`

**edge-length** none or int or float

Maximum visible arc length for centered parallel edge paths. none keeps the full shifted path.

Default: `none`

**edge-ratio** none or int or float

Maximum visible fraction of the base edge length for centered parallel edge paths. Combined with `edge-length` according to `edge-resolve-length`.

Default: `none`

**edge-resolve-length** string or function

Resolve `edge-length` and `edge-ratio`. Accepted string values are "min"/"shorter", "max"/"longer", "length"/"fixed", "ratio"/"relative", or "none"/"full". A function receives (`base-length`, `length`, `ratio`).

Default: `"min"`



**edge-accuracy** float

Arc-length accuracy for fitted parallel edge paths.

Default: 0.001

**edge-optimize** bool

Let Kurbo optimize the fitted parallel path.

Default: true

**source-style** dictionary or array or function or none

Source half-edge style dictionary, array of layer dictionaries, or callback. mark-position: "center-if-dangling" keeps an end marker at the paired-edge split point while centering it on dangling half edges. mark-orientation: "edge" makes a mark follow edge.orientation instead of raw path direction; reversed edges move the mark to the sink half and flip it, while undirected edges suppress it.

Default: ( : )

**sink-style** dictionary or array or function or none

Sink half-edge style dictionary, array of layer dictionaries, or callback. A callback receives edge data. mark-position: "center-if-dangling" has the same dangling-edge behavior as for source-style, and mark-orientation: "edge" participates in the same orientation-aware mark placement.

Default: ( : )

**edge-label** content or string or function or none

Edge label content or callback. A callback receives edge data.

Default: none

**edge-omega** float

Hobby curl used at the endpoints of paired edge curves.

Default: 1.0

**edge-trim-accuracy** float

Arc-length accuracy for trimming edge curves at node outsets.

Default: 0.001

**padding** none or int or float or array or dictionary

CeTZ canvas padding.

Default: 0.4

**debug-edge-radius** `int` or `float`

Radius for edge-position markers shown at debug  $\geq 2$ .

Default: `0.08`

**debug-edge-fill** `any`

Fill for edge-position markers shown at debug  $\geq 2$ .

Default: `rgb("#ff9f1c")`

**debug-edge-stroke** `any`

Stroke for edge-position markers shown at debug  $\geq 2$ .

Default: `rgb("#d72638") + 0.35pt`

**debug-edge-label-fill** `any`

Label fill for edge-position markers shown at debug  $\geq 2$ .

Default: `rgb("#7a1020")`

**subgraph-edge-style** `dictionary`

CeTZ style used to shade half-edges included in subgraph.

Default: (stroke: `rgb("#ffd166") + 4.5pt`)

**subgraph-edge-underlay** `bool`

Draw subgraph shading below the normal half-edge style.

Default: `true`

## subgraph

- `label()`
- `bits()`
- `compass()`
- `to-label()`
- `hedges()`
- `contains()`

## label

Construct an subgraph object from a base62 label.

```
#let g = graph.build({
 graph.node(<a>)
 graph.node()
 graph.edge(graph.source(<a>, compass: "e"),
graph.sink())
})
#let east = subgraph.compass(g, "e")
#let same = subgraph.label(g, subgraph.to-
label(east))
#subgraph.hedges(same).len()
```

1

## Parameters

```
label(
 graph: dictionary,
 label: string
) -> bytes
```

**graph** dictionary

Graph object whose half-edge set the label refers to.

**label** string

Base62 subgraph label returned by `to-label()` or produced by Linnest.

## bits

Construct an subgraph object from a boolean hedge array.

```
#let g = graph.build({
 graph.node(<a>)
 graph.node()
 graph.edge(graph.source(<a>), graph.sink())
})
#let first = subgraph.bits(g, (true, false))
#subgraph.contains(first, 0)
```

true

## Parameters

```
bits(
 graph: dictionary,
 bits: array
) -> bytes
```

**graph** dictionary

Graph object whose half-edge order defines the bit array.

**bits** array

Boolean array selecting half edges by graph half-edge index.

## compass

Construct an subgraph object from a DOT compass point such as "n" or "s".

```
#let g = graph.build({
 graph.node(<a>)
 graph.node()
 graph.edge(graph.source(<a>), compass: "e"),
 graph.sink())
})
#subgraph.hedges(subgraph.compass(g, "e")).len()
```

1

## Parameters

```
compass(
 graph: dictionary,
 compass: string
) -> bytes
```

**graph** dictionary

Graph object to filter by half-edge compass statement.

**compass** string

DOT compass point such as "n", "s", "e", or "w".

## to-label

Convert an subgraph object to its base62 label.

```
#let g = graph.build({
 graph.node(<a>)
 graph.node()
 graph.edge(graph.source(<a>), graph.sink())
})
#subgraph.to-label(subgraph.bits(g, (true,
false)))
```

1

## Parameters

```
to-label(subgraph: bytes) -> string
```

**subgraph** bytes

Subgraph object returned by this module or by graph.cycles/graph.forests.

## hedges

Return the hedge indices included in an subgraph object.

```
#let g = graph.build({
 graph.node(<a>)
 graph.node()
 graph.edge(graph.source(<a>), graph.sink())
})
#subgraph.hedges(subgraph.bits(g, (true,
false)))
```

(0,)

## Parameters

```
hedges(subgraph: bytes) -> array
```

**subgraph** bytes

Subgraph object to inspect.

## contains

Test whether an subgraph object includes a hedge.

```
#let g = graph.build({
 graph.node(<a>)
 graph.node()
 graph.edge(graph.source(<a>), graph.sink())
})
#subgraph.contains(subgraph.bits(g, (true,
false)), 0)
```

true

## Parameters

```
contains(
 subgraph: bytes ,
 hedge: int
) -> bool
```

**subgraph**    bytes

Subgraph object to inspect.

**hedge**    int

Half-edge index to test.